C Sharp 15 and Java 27

A comprehensive comparison of language features and functionality

Prepared • 25 October 2026

Executive assessment

C# 15 has the stronger overall set of language mechanisms for general-purpose application development. Its principal advantages are runtime-aware generics, user-defined value types, native nullability analysis, API constructs, interpretable expressions, and compiler-supported resumable workflows. C# 15 adds unions and closed class hierarchies, materially strengthening domain modeling.

Java 27 retains substantive strengths in rich enums, use-site variance, intersection types, final local bindings, sealed interfaces, declarative modules, and the default enforcement of exhaustive enhanced switches. Checked exceptions provide an additional enforceable contract, with corresponding composition costs. Virtual threads and preview structured concurrency strengthen Java functionality at the runtime and API level.

This is an independent technical assessment. The user's preference for C# is not evidence. The conclusion weights expressive power, compile-time assistance, semantic clarity, and implementation complexity. It does not establish that C# applications universally have fewer defects, run faster, or cost less to maintain.

Scope and method

The baseline is the documented C# 15 feature set and Java 27, assessed as of 25 October 2026. Microsoft identifies the .NET 11 SDK as prerelease. C# memory-safety changes require additional preview settings. Java primitive-pattern extensions and structured concurrency are identified as preview where discussed. Future proposals outside these documented feature sets receive no credit. [1–3]

Popularity, employment demand, ecosystem size, frameworks, IDE quality, licensing, deployment support, and benchmark rankings are excluded. Runtime-backed language semantics count where they explain a feature. Standard APIs are identified separately so that an API advantage is not mistaken for a language-syntax advantage.

Each domain presents comparable capabilities followed by an assessment and its limits. An advantage indicates a material benefit within that domain, not a numerical point. Features are not equally important, so counting winning rows would be misleading. Examples describe language behavior; this report does not include independently compiled examples or performance measurements.

Assessment by domain

Domain	Assessment	Reason
Core types and arithmetic	C#	More direct representation and arithmetic controls
Nullability and initialization	C# with qualifications	Built-in null-state analysis; Java has stronger final-binding mechanisms
Generics	C# overall	Reification and generic operators; Java leads in use-site variance and intersection casts
Object-oriented APIs	C#	More direct API concepts and composition mechanisms

Domain	Assessment	Reason
Data and closed domains	C# overall	Unions and representation choices; Java retains rich enums and sealed interfaces
Patterns and control flow	Split	C# breadth; Java stricter default exhaustiveness
Functions and code as data	C# overall	Local functions and compiler-generated expression trees
Iteration and concurrency	Split across layers	C# resumable syntax; Java virtual threads and preview task scopes
Errors and cleanup	Split	Different contracts, filtering, asynchronous cleanup and failure preservation
Memory and native access	C# for direct language control	Both platforms have capable native APIs
Everyday ergonomics	C# overall	More built-in construction and call-site syntax
Modules and source composition	Split	Java module contracts; C# partial declarations and dynamic syntax

1 Core types and numeric semantics

Representation choices affect copying, identity, arithmetic policy, and generic storage.

Feature	C# 15	Java 27	Assessment
User-defined values	Structs and record structs alongside reference classes.	Primitives are values; ordinary classes and records are references.	C# provides more control.
Integral types	Signed, unsigned, and native-sized integers.	Primarily signed integers; unsigned char and additional unsigned APIs.	C# has broader native support.
Decimal arithmetic	Built-in fixed-precision decimal, literals and operators.	Arbitrary-precision BigDecimal API.	C# syntax; Java precision range.
Overflow	checked and unchecked control integral overflow policy.	Ordinary arithmetic wraps; exact arithmetic methods can detect overflow.	C# language integration.
Parameters	By value by default; explicit ref, out and readonly-reference mechanisms.	Always passes values, including copies of object references.	C# flexibility; Java uniformity.

Neither language detects all integral overflow by default. BigDecimal and decimal are different numeric models. C# value types do not imply stack allocation or an automatic performance win. Both languages can mutate an object through a passed reference; C# additionally supports passing the caller's variable by reference.

Sources [4–8].

2 Nullability and initialization

The relevant distinction is the guarantee established by each construct, rather than its name.

Feature	C# 15	Java 27	Assessment
Reference nullability	Nullable annotations and compiler null-state analysis.	Reference types permit null; annotations alone impose no core nullness checking.	C# native analysis advantage.
Null handling	Null-conditional access and coalescing operators.	Explicit checks, conditional expressions and Optional APIs.	C# concise expression.
Optional values	T? for built-in and user-defined value types.	Nullable wrappers and optional-value APIs.	C# consistent language syntax.
Construction obligations	required members and init-only accessors.	Constructor parameters and definite assignment of blank final fields.	Different construction models.
Final bindings	readonly in selected contexts; no general final modifier for ordinary locals.	final locals, parameters and fields.	Java binding immutability.

Nullable reference types generate diagnostics and do not change runtime types. required establishes ordinary construction-site obligations, not business validity. readonly and final references do not freeze referenced objects. Default-initialized C# structs can bypass constructor validation. Java's blank-final rules enforce assignment and prevent reassignment.

Sources [9–13].

3 Generics and generic algorithms

Generic capability includes runtime identity, representation, constraints, variance and available operations.

Feature	C# 15	Java 27	Assessment
Runtime identity	Constructed generic arguments remain available at runtime.	Parameterized types generally undergo erasure; declarations can retain metadata.	C# fewer runtime restrictions.
Value arguments	List<int> stores elements without per-element boxing.	List<Integer>; primitives are not generic arguments.	C# representation advantage.
Generic construction	new T() with a suitable constraint; arrays of T.	No direct new T() or new T[]; factories and other mechanisms are used.	C# implementation advantage.
Variance	Declaration-site in and out on interfaces and delegates for reference arguments.	Use-site extends and super wildcards.	Java flexibility for consuming generic classes.
Static contracts	Static abstract interface members support operator and static-method requirements.	Bounds express class and interface relationships; strategies supply operations.	C# generic algorithm advantage.

Feature	C# 15	Java 27	Assessment
Intersection types	Multiple interface constraints, without general intersection-cast syntax.	Intersection bounds and casts.	Java advantage in this dimension.

C# leads overall through reification, value-type support and static polymorphism. Java's wildcard flexibility is substantive. Erasure does not eliminate Java's compile-time generic type checking or every piece of generic declaration metadata. Mutable generic list types are not freely covariant in either language.

Sources [4, 14–17].

4 Object oriented APIs

The languages share a class and interface foundation, but C# gives more API concepts dedicated syntax.

Feature	C# 15	Java 27	Assessment
Inheritance	Single class inheritance, multiple interfaces and default interface members.	Broadly comparable mechanisms.	Broad parity.
Overriding	Ordinary class methods are non-virtual by default; virtual and abstract expose extension points.	Overridable instance methods are the usual default; final prevents overriding.	Different extension policies.
Interface-specific behavior	Explicit interface implementations can separate identical member signatures.	Shared implementations or adapters normally provide alternatives.	C# advantage.
Properties and indexing	Dedicated properties and indexers, including automatic accessors.	Methods and conventions.	C# direct expression.
Extensions	Extension methods, properties, operators and C# 15 indexers.	Static helpers, adapters or changes to declarations.	C# API composition advantage.
Events	Dedicated subscription members with publisher-controlled notification.	Listener and observer interfaces and methods.	C# native support.
Operators	User-defined operators and conversions.	No general user-defined operator overloading.	C# expression; Java fixed semantics.

C# more directly expresses API intent. Its overriding default makes extension permission explicit, while Java makes extensibility easier. Extensions do not alter the original type or grant private access. Operator overloading improves domain notation when meanings remain predictable; surprising conversions and operators add reading cost.

Sources [18–23].

5 Data and closed domains

C# 15 substantially improves composition of alternative outcomes, while Java retains useful restrictions.

Feature	C# 15	Java 27	Assessment
Records	Record classes and structs; record-class inheritance and value-oriented behavior.	Final reference records with generated accessors, equality and formatting.	C# options; Java narrower abstraction.
Immutability	Readonly structs and fields, init accessors; records may be mutable.	Final components and fields.	Neither implies deep immutability.
Small aggregates and copies	Tuples, deconstruction and with expressions.	Named records, constructors and methods.	C# concise composition.
Unions	Named closed sets of existing case types without shared inheritance.	Sealed hierarchies and wrappers model alternatives.	C# composition advantage.
Closed inheritance	closed classes restrict direct subtypes to their assembly.	Sealed classes and interfaces specify permitted subtypes.	Split; Java offers sealed interfaces.
Enums	Numeric types support codes and flags; unnamed values are possible.	Constants can carry state, implement interfaces and provide behavior.	Java rich-domain advantage.
Equality	Records supply value-oriented ==; other types can overload it.	Reference == tests identity; records provide value-oriented equals.	Natural notation versus uniform identity distinction.

C# with expressions are shallow copies, and neither language's records automatically freeze reachable objects. Generated C# unions store object? contents, box value-type cases and can contain null when default-initialized. closed constrains direct descendants; further inheritance can remain open. Java permits deliberate reopening through non-sealed subtypes. C# leads in modeling options, with important Java exceptions.

Sources [24–30].

6 Patterns and control flow

Pattern breadth and enforcement should be judged independently.

Feature	C# 15	Java 27	Assessment
Pattern vocabulary	Type, property, positional, relational, logical and list patterns.	Type and record patterns, guards, plus preview primitive patterns.	C# breadth advantage.
Deconstruction	Positional patterns can use a suitable Deconstruct method.	Record patterns decompose records, including nested records.	C# general adaptation; both support nesting.
Closed-set coverage	Recognizes exhaustive unions and closed-hierarchy switch expressions.	Coverage of enums, sealed hierarchies and record-pattern combinations.	Both have substantive analysis.
Enforcement	Incomplete switch expressions generally warn; ordinary switches need not be exhaustive.	Incomplete switch expressions and enhanced switch statements are errors.	Java default enforcement advantage.

Feature	C# 15	Java 27	Assessment
Nested-loop exits	C# 15 adds labeled break and continue.	Longstanding labeled break and continue.	Broad parity for loop control.

C# has a richer pattern vocabulary. Java has stricter default enforcement for enhanced switches. C# projects can elevate relevant warnings to errors. Older non-enhanced Java switch statements need not be exhaustive. A catch-all branch can also conceal newly added cases, so exhaustive modeling depends on how switches are written.

Sources [1–3, 31–32].

7 Functions queries and code as data

Both support passing behavior; C# provides more mechanisms around that foundation.

Feature	C# 15	Java 27	Assessment
Anonymous functions	Lambdas and method groups target delegates.	Lambdas and method references target functional interfaces.	Broad parity in passing behavior.
Capture	Mutable locals can be captured; static lambdas prohibit capture.	Captured locals must be final or effectively final.	C# flexibility; Java stricter bindings.
Local functions	Named local helpers, including generic, async and iterator functions.	Lambdas and local classes provide alternatives.	C# direct support.
Queries	Language query syntax translates to query-pattern method calls.	Stream pipelines use APIs and lambdas.	C# language integration.
Expression trees	Supported expression lambdas can be captured as inspectable code structures.	No comparable general compiler conversion of ordinary lambdas into expression trees.	C# interpretable API advantage.

Java's capture restriction does not make captured objects immutable or thread-safe. C# static lambdas explicitly prevent capture. Expression trees are more consequential than query notation alone because APIs can inspect supplied expressions, but compiler capture supports a subset of C# syntax. Query syntax does not guarantee arbitrary provider translation or efficient execution.

Sources [33–36].

8 Iteration asynchronous execution and concurrency

Language-supported suspension and runtime-supported lightweight threads are different mechanisms.

Feature	C# 15	Java 27	Assessment
Generators	yield return generates resumable iterator state.	Iterator and stream producers use APIs and implementations.	C# compiler support.
Async methods	async and await support sequential-looking asynchronous workflows.	Futures and completion stages provide API composition.	C# language integration.

Feature	C# 15	Java 27	Assessment
Async sequences	Async iterators and await foreach.	API-based approaches without equivalent iterator syntax.	C# language advantage.
Blocking-style concurrency	Often distinguishes synchronous and asynchronous operations.	Virtual threads support many blocking-style I/O tasks over fewer platform threads.	Java runtime strength.
Task lifetimes	Task composition alone does not impose lexical ownership of child tasks.	Preview StructuredTaskScope groups tasks and bounds their lifetimes.	Java preview API advantage.
Cancellation	Cooperative cancellation tokens.	Interruption and related task APIs.	Both depend on cooperation.
Mutual exclusion	Language lock and runtime locking mechanisms.	Language synchronized and monitors.	Broad parity for basic exclusion.

C# provides stronger syntax and compiler support for resumable workflows. await suspends an async method rather than blocking its evaluating thread. Java virtual threads can preserve ordinary blocking call chains, especially for I/O-heavy concurrency. Structured concurrency adds task-lifetime organization but remains preview. Neither approach establishes a general CPU-speed advantage or proves programs free of races and deadlocks.

Sources [3, 37–42].

9 Error contracts and resource management

Different mechanisms provide different guarantees; this domain has no unconditional winner.

Feature	C# 15	Java 27	Assessment
Checked failures	Thrown exception types are not enforced in signatures.	Checked exceptions require catch-or-declare handling.	Java enforceable contract; composition cost.
Returned outcomes	Unions directly compose result alternatives.	Sealed result hierarchies model alternatives.	C# syntax and composition.
Catch policies	Boolean when filters.	Multi-catch; further tests normally run in handlers.	Different useful mechanisms.
Scoped cleanup	using statements and declarations.	Try-with-resources.	Broad parity.
Async cleanup	Dedicated await using.	AutoCloseable supplies a synchronous closing contract.	C# explicit async support.
Competing failures	Disposal failures can replace an already propagating failure without explicit handling.	Cleanup failures are automatically retained as suppressed exceptions.	Java preservation advantage.

Returning a C# union does not constrain all exceptions a method can throw. Java checked contracts likewise do not enumerate unchecked exceptions. C# exception filters run before stack unwinding, unlike conditions inside a handler. Java suppressed exceptions address a separate correctness concern during cleanup.

Sources [27, 32, 43–45].

10 Memory control and native access

C# provides more direct language control; modern Java also has substantial native-access APIs.

Feature	C# 15	Java 27	Assessment
Managed memory	Garbage collection with user-defined value representations.	Garbage collection with primitive and object representations.	No general speed verdict.
Stack allocation	Explicit stackalloc, including safe use through spans.	No equivalent general source-level stack-allocation expression.	C# allocation control.
Lifetimes	ref struct restrictions prevent certain heap escapes.	Foreign-memory arenas and segments control access and lifetime through APIs.	Compiler versus API and runtime controls.
Pointers	Opt-in pointers and function pointers; preview revised safety obligations.	Foreign-memory APIs rather than ordinary pointer syntax.	C# low-level expression.
Native functions	P/Invoke and generated bindings.	Foreign Function and Memory API, alongside JNI.	Both capable; workload-dependent suitability.

Value semantics, spans and stack allocation offer options rather than universal allocation or speed guarantees. C# lifetime restrictions do not amount to a general ownership proof. Java should receive credit for current foreign-memory and function APIs, rather than being evaluated only through JNI. Native calls introduce safety obligations on both platforms.

Sources [1, 46–49].

11 Everyday syntax and construction

Convenience features reduce ceremony, with importance determined by actual usage.

Feature	C# 15	Java 27	Assessment
Inference	var infers a static local type.	var infers a static local type.	Broad parity.
Call sites	Named and optional arguments.	Overloads, configuration objects and builders.	C# direct expression.
Strings	Interpolation, raw strings and handler-based customization.	Text blocks and formatting APIs.	C# language integration.
Collections	Target-typed expressions, spreads and C# 15 constructor or builder arguments.	Array initializers and collection factory APIs.	C# general construction syntax.
Constructors	Primary constructors on ordinary classes and structs, plus records.	Concise record construction and flexible constructor bodies.	Different conveniences; broader C# shorthand.
Small programs	Top-level statements.	Compact source files and instance main methods.	Both reduce ceremony.

C# primary-constructor parameters do not automatically become public properties on ordinary classes. Java now permits suitable constructor statements before explicit superclass invocation and supports low-ceremony small programs. These current capabilities prevent outdated comparisons based on historical boilerplate. Convenience is meaningful but not equivalent to greater computational power.

Sources [1–2, 50–54].

12 Modules metadata and source composition

Java modules and C# source composition solve distinct architectural needs.

Feature	C# 15	Java 27	Assessment
Internal boundaries	Assembly-scoped internal, combined access scopes and friend assemblies; namespaces organize names.	Package access and named modules provide distinct boundaries.	Different boundary models.
Module contracts	No directly equivalent language declaration for package exports, reflective opening and dependencies.	requires, exports, opens and service relationships.	Java declarative modularity advantage.
Metadata	Attributes and reflection.	Annotations and reflection, including retention policies.	Broad parity in purpose.
Dynamic binding	dynamic defers selected binding checks to runtime.	Reflection and method-handle APIs, without equivalent ordinary dynamic-member syntax.	C# direct expression with fewer static checks.
Source composition	Partial types and members combine generated and handwritten declarations.	Generated additional or companion types.	C# language composition advantage.
Generation	Compiler-platform source generators.	Annotation-processing APIs.	Both capable compiler and API facilities.

Java provides a more explicit language vocabulary for module dependencies, exports and reflective access. C# assemblies and friend access are useful but not interchangeable with that model. C# offers stronger direct source composition. Both support metadata-driven generation, so attributing that entire capability to either language would be inaccurate.

Sources [18, 55–59].

Implications for language evaluation

The strongest case for C# is the combined availability of direct data representation, runtime-aware generic abstraction, native null-state analysis, API-specific constructs, expression capture, and compiler-supported suspension. C# 15 adds closed alternatives through both inheritance and composition. These mechanisms frequently remove supporting code that Java must express with additional methods, interfaces, wrappers, or state management.

The strongest case for Java concerns selected semantics and contracts. Final bindings, rich enums, sealed interfaces, stricter exhaustive switches and explicit modules can constrain designs in useful ways. Checked exceptions make one category of failure contract explicit, although their propagation can complicate generic and functional composition. Virtual threads provide a practical runtime alternative to asynchronous call chains.

Maintainability should be evaluated against the permitted language subset and enforcement policy. More features create opportunities to express intent and additional interactions to understand. A smaller surface also retains complexity through erasure, wildcard capture, construction rules and exception propagation. Neither feature inventory alone establishes lifetime maintenance cost.

The assessment is qualitative and source-based. It does not measure execution speed, security incident rates, defect frequency, developer learning time, or team delivery performance. A concrete selection decision should identify the required guarantees, representative workflows, memory constraints, and allowed language constructs before applying these domain judgments.

Source references

Primary sources from Microsoft, Oracle and the official Java developer documentation. URLs correspond to sources reviewed during the evaluation. Version and preview status are anchored to 25 October 2026. Number ranges refer to every source in the stated range. The linked documents may evolve after this report date.

[1] C# 15 feature set

https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-15

[2] Java language changes

https://docs.oracle.com/en/java/javase/27/language/java-language-changes-summary.html

[3] Java 27 changes and previews

https://docs.oracle.com/en/java/javase/27/migrate/significant-changes-jdk-27-release.html

[4] Core type models

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/struct

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-4.html

[5] Integral types

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/integral-numeric-types

[6] Decimal models

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/math/BigDecimal.html

[7] Overflow behavior

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/checked-and-unchecked

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/lang/Math.html

[8] Method arguments

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/method-parameters

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-15.html

[9] Nullable references

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/null-safety/nullable-reference-types

[10] Optional values

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/nullable-value-types

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/util/Optional.html

[11] Initialization members

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/required

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/init

[12] Definite assignment

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-16.html

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/variables

[13] Readonly bindings

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/readonly

[14] C# generic types

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/generics

[15] Java generic restrictions

https://dev.java/learn/language/fp/generics/restrictions/

[16] Variance

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/in-generic-modifier

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/out-generic-modifier

https://dev.java/learn/language/fp/generics/wildcards/

[17] Generic constraints and static members

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/where-generic-type-constraint

https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/interface-implementation/static-virtual-interface-members

[18] Java interfaces and annotations

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-9.html

[19] Overriding policies

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/virtual

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-8.html

[20] Explicit interface implementation

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/interfaces/explicit-interface-implementation

[21] Properties and indexers

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/properties

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/indexers/

[22] Extension members

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/extension-methods

[23] Events and operators

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/event

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/operator-overloading

[24] C# records

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/record

[25] Java records

https://docs.oracle.com/en/java/javase/27/language/records.html

[26] Tuples

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/value-tuples

[27] Union types

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/union

[28] Closed hierarchies

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/closed

https://docs.oracle.com/en/java/javase/27/language/sealed-classes-interfaces.html

[29] Enum models

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/enum

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-8.html#jls-8.9

[30] Equality

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/equality-operators

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-15.html#jls-15.21

[31] Pattern matching

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/patterns

https://docs.oracle.com/en/java/javase/27/language/pattern-matching-switch.html

[32] Java statements and cleanup

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-14.html

[33] Lambdas and capture

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/lambda-expressions

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-15.html#jls-15.27

[34] Local functions

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/building-software/local-functions

[35] Query expressions and streams

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/expressions

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/util/stream/package-summary.html

[36] Expression trees

https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/expression-trees/

[37] Iteration and suspension

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/yield

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/await

[38] Futures

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/util/concurrent/CompletableFuture.html

[39] Virtual threads

https://docs.oracle.com/en/java/javase/27/core/virtual-threads.html

[40] Structured concurrency preview

https://docs.oracle.com/en/java/javase/27/core/structured-concurrency.html

[41] Cancellation

https://learn.microsoft.com/en-us/dotnet/standard/threading/cancellation-in-managed-threads

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/lang/Thread.html

[42] Synchronization

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/lock

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-17.html

[43] Java exception contracts

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-11.html

[44] C# exceptions and filters

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/exception-handling-statements

[45] C# disposal

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/using

[46] Stack allocation

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/stackalloc

[47] Restricted lifetimes

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/ref-struct

[48] Foreign memory and functions

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/lang/foreign/package-summary.html

[49] Platform invoke

https://learn.microsoft.com/en-us/dotnet/standard/native-interop/pinvoke

[50] Named and optional arguments

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/named-and-optional-arguments

[51] Strings

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/tokens/interpolated

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/tokens/raw-string

https://docs.oracle.com/en/java/javase/27/language/text-blocks.html

[52] Collection construction

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/collection-expressions

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/util/List.html

[53] Constructors

https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/tutorials/primary-constructors

https://docs.oracle.com/en/java/javase/27/language/flexible-constructor-bodies.html

[54] Small programs

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/program-structure/top-level-statements

https://docs.oracle.com/en/java/javase/27/language/compact-source-files-instance-main-methods.html

[55] Modules

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-7.html

[56] Assembly access

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/internal

https://learn.microsoft.com/en-us/dotnet/standard/assembly/friend

[57] Attributes and reflection

https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/reflection-and-attributes/

[58] Dynamic binding

https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/interop/using-type-dynamic

[59] Source composition and generation

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/partial-classes-and-methods

https://docs.oracle.com/en/java/javase/27/docs/api/java.compiler/javax/annotation/processing/package-summary.html